Search Typeahead System

High-Level Design — Final Project Report

Table of Contents

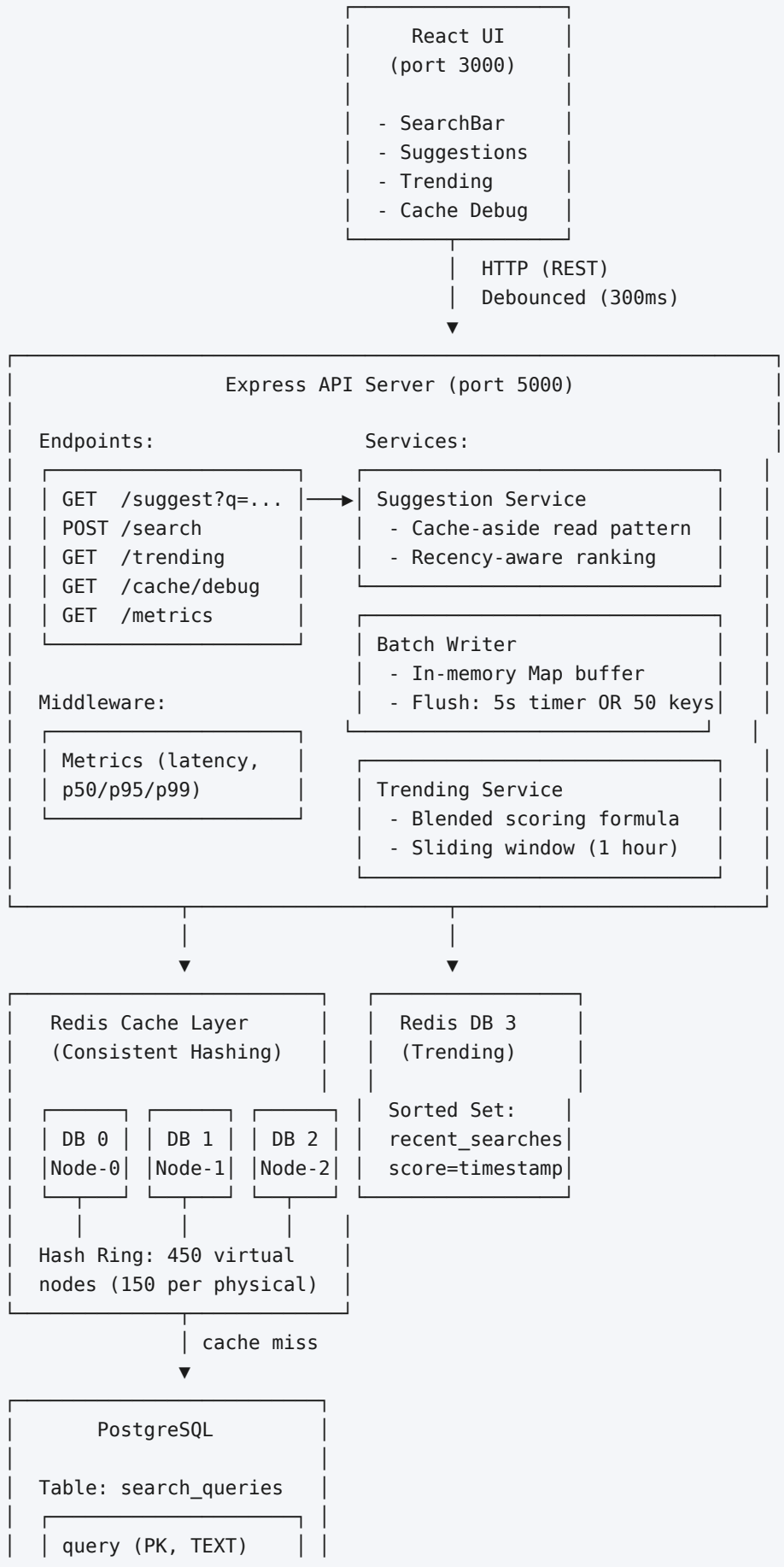
---

1. Architecture

1.1 High-Level Overview

The system is a full-stack search typeahead application that provides real-time prefix-based suggestions, trending search rankings, and optimized write handling. It consists of four layers: a React frontend, a Node.js/Express API server, a Redis distributed cache, and a PostgreSQL database.

1.2 Architecture Diagram



```
  │  │ count (BIGINT)      │ │
  │  │ last_searched_at    │ │
  │  └─────────────────────┘ │
  │                          │
  │  Index: idx_query_prefix │
  │  (text_pattern_ops)      │
  └──────────────────────────┘
```

1.3 Data Flow: Read Path (Suggestion Lookup)

```
User types "iph" → [300ms debounce] → GET /suggest?q=iph

  1. Hash "iph" using MD5 → e.g., hash = 0xa3f2b891
  2. Binary search on consistent hash ring → lands on node-2 (Redis DB 2)
  3. Check Redis DB 2 for key "suggest:iph"
     ├── HIT  → Return cached JSON array (top 10 suggestions)
     └── MISS → Query PostgreSQL:
                  SELECT query, count FROM search_queries
                  WHERE query LIKE 'iph%'
                  ORDER BY count DESC LIMIT 20
              → Apply recency-aware ranking (blended score)
              → Take top 10
              → Store in Redis DB 2 with 60s TTL
              → Return to client
```

1.4 Data Flow: Write Path (Search Submission)

```
User submits "iphone 15" → POST /search

  1. Record in Redis Sorted Set (DB 3):
     ZADD recent_searches <timestamp> "iphone 15"
     Prune entries older than 1 hour:
     ZREMRANGEBYSCORE recent_searches 0 <cutoff>

  2. Add to Batch Writer buffer: Map { "iphone 15" → count + 1 }
  3. Return { "message": "Searched" } immediately

  ... Later (on flush timer or buffer full) ...

  4. Batch Writer flushes:
     INSERT INTO search_queries (query, count)
     VALUES ('iphone 15', 7), ...
     ON CONFLICT (query)
     DO UPDATE SET count = count + EXCLUDED.count

  5. Invalidate cache: Delete keys
     "suggest:i", "suggest:ip", ..., "suggest:iphone 15"
```

1.5 Component Summary

Component	Technology	Purpose
Frontend	React	Search input with debouncing, suggestion dropdown, trending display, cache debug panel
API Server	Express.js	REST endpoints, middleware for metrics
Suggestion Service	Node.js	Cache-aside reads, recency ranking
Batch Writer	Node.js (in-memory Map)	Aggregates writes, flushes periodically
Trending Service	Node.js + Redis Sorted Set	Tracks recent activity, computes blended scores
Cache Layer	Redis (DB 0, 1, 2)	Distributed cache with consistent hashing
Trending Store	Redis (DB 3)	Sorted Set for sliding window recent searches
Database	PostgreSQL	Persistent storage with prefix-optimized index

2. Dataset Source and Loading Instructions

2.1 Dataset Source

The system uses the **AOL Search Query Logs** dataset, a real-world search query dataset originally released by AOL Research in 2006. It contains approximately 20 million web search queries collected from ~650,000 users over a 3-month period.

- **Source:** AOL Research / Available on Kaggle
- **Format:** Tab-separated text files with columns: `AnonID`, `Query`, `QueryTime`, `ItemRank`, `ClickURL`
- **Raw size:** ~226 MB (3.6 million lines per file)
- **After processing:** ~1.24 million unique queries with aggregated search counts

2.2 Data Processing Pipeline

The loading script (`backend/src/scripts/loadDataset.js`) performs:

1. **Parsing:** Streams AOL `.txt` files line-by-line using Node.js `readline` to handle large files without loading them entirely into memory
2. **Extraction:** Takes the `Query` column (tab-separated, index 1), normalizes to lowercase
3. **Aggregation:** Uses an in-memory `Map<query, count>` to count occurrences of each query
4. **Filtering:** Removes queries shorter than 2 characters and longer than 200 characters
5. **Batch loading:** Inserts into PostgreSQL in batches of 1,000 rows using multi-row `INSERT ... ON CONFLICT DO UPDATE`

2.3 Loading Instructions

```
# Step 1: Download the AOL Search Query Logs from Kaggle
#         Search for "AOL search query logs" on kaggle.com
#         Download and extract the .txt files

# Step 2: Place the files in the data directory
mkdir -p data/aol-search
# Copy/move the extracted .txt files into data/aol-search/

# Step 3: Run the loader
cd backend
npm run load-data
```

The loader auto-detects the dataset location in this priority:

1. `data/aol-search/*.txt` (AOL format)

2. `data/AOL-user-ct-collection/*.txt` (alternate directory name)
3. `data/*.txt` (any .txt in data root)
4. `data/dataset.csv` (CSV fallback with `query,count` columns)

2.4 PostgreSQL Schema

```
CREATE TABLE search_queries (
    query            TEXT PRIMARY KEY,
    count            BIGINT NOT NULL DEFAULT 0,
    last_searched_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_query_prefix
ON search_queries (query text_pattern_ops);
```

- `TEXT PRIMARY KEY` — each unique query string is a row
- `BIGINT count` — total number of times this query was searched
- `text_pattern_ops` index — enables B-tree index usage for `LIKE 'prefix%'` queries

---

3. API Documentation

3.1 GET /suggest?q=<prefix>

Returns up to 10 prefix-matching suggestions, ranked by a blended score of total count and recent activity.

Request

```
GET /suggest?q=iphone
```

Response

```
{
  "suggestions": [
    { "query": "iphone coupon", "count": "605638", "score": 0.6, "recentCount": 0 },
    { "query": "iphone setup", "count": "93923", "score": 0.093, "recentCount": 0 },
    { "query": "iphone case", "count": "45210", "score": 0.045, "recentCount": 3 }
  ]
}
```

Field	Description
`query`	The matching search query string
`count`	Total all-time search count from PostgreSQL
`score`	Blended ranking score (0.6 * total + 0.4 * recent)
`recentCount`	Number of times searched in the last 1 hour

Behavior:

- Empty or whitespace-only prefix returns an empty array
- Input is normalized to lowercase
- Fetches top 20 from DB, applies recency ranking, returns top 10
- Results are cached in Redis with 60s TTL

3.2 POST /search

Submits a search query. The query is buffered for batch writing to PostgreSQL and recorded in the trending service.

Request

```
POST /search
Content-Type: application/json

{ "query": "iphone 15 pro" }
```

Response

```
{ "message": "Searched" }
```

Behavior:

- Query is immediately added to the batch writer's in-memory buffer
- Query is recorded in the Redis Sorted Set for recency tracking
- The actual DB write happens on the next flush (timer-based or size-based)
- Returns immediately without waiting for the DB write

3.3 GET /trending

Returns the top 10 trending searches, ranked using recency-aware blended scoring.

Request

```
GET /trending
```

Response

```
{
  "trending": [
    { "query": "google", "count": "977153", "score": 0.6, "recentCount": 0 },
    { "query": "youtube", "count": "850421", "score": 0.52, "recentCount": 5 }
  ]
}
```

3.4 GET /cache/debug?prefix=<prefix>

Debug endpoint showing the consistent hashing routing for a given prefix.

Request

```
GET /cache/debug?prefix=iphone
```

Response

```
{
  "prefix": "iphone",
  "hashValue": "b3f45b2",
  "assignedNode": "node-1",
  "redisDb": 1,
  "cacheHit": true,
  "totalNodes": ["node-0", "node-1", "node-2"]
}
```

Field	Description
`hashValue`	MD5 hash of the prefix (first 8 hex chars converted to integer, displayed as hex)
`assignedNode`	The node selected by clockwise walk on the hash ring
`redisDb`	The Redis database number corresponding to that node
`cacheHit`	Whether the key currently exists in that Redis node
`totalNodes`	All nodes in the consistent hash ring

3.5 GET /metrics

Returns system performance metrics.

Response

```
{
  "totalRequests": 164,
  "suggest": {
    "count": 114, "p50": "1.10ms", "p95": "113.61ms", "p99": "116.20ms"
  },
  "overall": {
    "p50": "1.10ms", "p95": "112.37ms", "p99": "116.20ms"
  },
  "cache": {
    "hits": 95, "misses": 19, "hitRate": "83.33%"
  },
  "batchWriter": {
    "bufferSize": 0, "totalSubmissions": 50, "totalFlushes": 1,
    "totalDbWrites": 5, "totalWritesSaved": 45, "writeReduction": "90.00%"
  }
}
```

---

4. Design Choices and Trade-offs

4.1 Data Storage: PostgreSQL with `text_pattern_ops`

Choice: PostgreSQL with a B-tree index using the `text_pattern_ops` operator class.

How it works: The `text_pattern_ops` operator class tells PostgreSQL to build the B-tree index in a way that supports pattern matching with `LIKE 'prefix%'`. Without it, PostgreSQL would fall back to a sequential scan for LIKE queries, which is O(n) over all rows.

Why not an in-memory Trie?

Aspect	PostgreSQL + text_pattern_ops	In-Memory Trie
Lookup speed	~1-5ms (indexed B-tree)	~0.01ms (pointer traversal)
Persistence	Survives restart	Lost on crash
Memory usage	Disk-backed, low memory	All data in RAM
Scalability	Handles millions of rows	Limited by available RAM
ACID compliance	Full transactions	None
Concurrent access	Built-in locking	Must implement manually

Trade-off: We sacrifice sub-millisecond lookup speed for persistence, ACID compliance, and the ability to handle 1.2M+ queries without memory pressure. The Redis cache layer compensates for the speed difference — cached lookups are <1ms.

4.2 Distributed Cache: Consistent Hashing

Choice: MD5-based consistent hashing with 150 virtual nodes per physical node, distributing cache keys across 3 Redis database instances.

4.2.1 Why Consistent Hashing over Modulo Hashing

With **modulo hashing** (`hash(key) % N`), if you add or remove a node (N changes), nearly every key maps to a different node — causing a cache stampede where all keys miss simultaneously.

With **consistent hashing**, nodes are placed on a circular ring. Each key hashes to a point on the ring and is assigned to the next node clockwise. When a node is added or removed, only `K/N` keys (where K = total keys, N = nodes) need to remap.

```
Modulo:     N changes from 3 → 4, ~75% of keys remap
Consistent: N changes from 3 → 4, ~25% of keys remap
```

4.2.2 Why Virtual Nodes (150 per physical node)

Without virtual nodes, 3 physical nodes create only 3 points on the ring, leading to uneven distribution — one node might handle 60% of keys while another handles 10%.

Virtual nodes create 150 points per physical node (450 total), spreading each node's responsibility evenly across the ring. This ensures roughly equal load distribution.

Trade-off: More virtual nodes = more even distribution but more memory for the ring and slightly slower lookups (larger sorted array to binary search). 150 is a practical sweet spot.

4.2.3 Cache-Aside Pattern

```
Read:  App → Check Cache → [HIT] → Return
                         → [MISS] → Query DB → Write to Cache → Return

Write: App → Write to DB → Invalidate Cache (delete keys)
```

Why cache-aside over write-through? Cache-aside only populates the cache for data that is actually requested. Write-through would cache every DB write, including queries that are never looked up — wasting cache memory.

Why invalidation (delete) over update? A single query like "iphone" affects multiple cache keys (`suggest:i`, `suggest:ip`, `suggest:iph`, ..., `suggest:iphone`). Each key stores a pre-computed top-10 list. Updating all these lists correctly would require re-querying and re-ranking — more expensive than simply deleting them and letting the next read repopulate.

4.2.4 TTL (60 seconds)

TTL	Cache Hit Rate	Data Freshness
Short (10s)	Lower — keys expire quickly	High — stale data for at most 10s
Long (300s)	Higher — keys stay cached	Lower — user might see 5-minute-old rankings
60s (chosen)	Balanced	Acceptable staleness for search suggestions

4.2.5 Simulated Nodes via Redis Databases

Rather than running 3 separate Redis instances, we use Redis databases 0, 1, and 2 as logical cache nodes, and database 3 for trending. This demonstrates the consistent hashing concept without operational overhead. In production, these would be separate Redis instances or a Redis Cluster.

4.3 Trending Searches: Recency-Aware Ranking

Choice: Blended scoring formula combining all-time popularity with recent activity.

4.3.1 The Formula

```
score = 0.6 x normalized_total_count + 0.4 x normalized_recent_count

Where:
  normalized_total_count  = query_count / max_count_among_candidates
  normalized_recent_count = recent_searches_in_1hr / max_recent_among_candidates
```

4.3.2 Why Blended Scoring

Approach	Problem
Pure count-based (`ORDER BY count DESC`)	Permanently favors historically popular queries. A query searched 1M times 5 years ago always outranks a query trending right now.
Pure recency-based	Too volatile. A query searched 3 times in the last hour outranks everything, even if it's irrelevant.
Blended (chosen)	Historically popular queries maintain a baseline rank (60% weight), but currently trending queries get a boost (40% weight).

4.3.3 Worked Example

Query	All-time Count	Recent (1hr)	Norm. Total	Norm. Recent	Score
google	500,000	0	1.00	0.00	**0.60**
chatgpt	50,000	15	0.10	1.00	**0.46**
youtube	400,000	3	0.80	0.20	**0.56**

Without recency: google > youtube > chatgpt. With recency: same order, but "chatgpt" jumps closer to the top because it's actively being searched right now.

4.3.4 Sliding Window

Recent searches are tracked in a Redis Sorted Set where the score is the timestamp:

```
ZADD recent_searches <timestamp_ms> "chatgpt"
```

On every new search, entries older than 1 hour are pruned:

```
ZREMRANGEBYSCORE recent_searches 0 <now - 3600000>
```

This creates a sliding window — old activity drops off automatically without manual cleanup.

Trade-off: Short window (15 min) = more responsive to trends but more volatile. Long window (6 hours) = smoother rankings but slower to react. 1 hour is a practical middle ground.

4.4 Batch Writes

Choice: In-memory `Map<string, number>` buffer that aggregates search submissions and flushes to PostgreSQL on a timer (every 5s) or size threshold (50 unique queries).

4.4.1 How It Works

```
Search: "google"  → Buffer: { google: 1 }
Search: "google"  → Buffer: { google: 2 }
Search: "amazon"  → Buffer: { google: 2, amazon: 1 }
Search: "google"  → Buffer: { google: 3, amazon: 1 }

... 5 seconds pass (timer triggers flush) ...

Flush: Single SQL statement:
  INSERT INTO search_queries (query, count)
  VALUES ('google', 3), ('amazon', 1)
  ON CONFLICT DO UPDATE SET count = count + EXCLUDED.count

Result: 4 submissions → 2 DB writes (50% reduction)
```

4.4.2 Why Batch Instead of Direct Writes

Aspect	Direct Write	Batch Write
DB writes per search	1	1/N (where N = avg submissions per unique query)
Latency of POST /search	Depends on DB	Instant (buffered)
Data loss risk	None	Buffered data lost on crash
DB connection pressure	High under load	Low — one connection per flush

4.4.3 Failure Trade-off

If the application crashes before a flush, all buffered writes are lost. Possible mitigations (discussed, not implemented):

1. **Write-ahead log (WAL) to disk:** Write each submission to a local file before buffering. On restart, replay the log. Adds disk I/O overhead.
2. **Redis as intermediate buffer:** Use `HINCRBY` to buffer in Redis instead of in-memory. Survives app crash but not Redis crash.
3. **Smaller flush interval:** Reduce from 5s to 1s. Less data loss but more DB writes.

The current design accepts the small data loss risk in exchange for significantly reduced DB pressure.

4.4.4 Cache Invalidation on Flush

After flushing to PostgreSQL, the batch writer invalidates all cache keys for the affected queries' prefixes. For example, flushing "iphone" invalidates `suggest:i`, `suggest:ip`, `suggest:iph`, ..., `suggest:iphone`. This ensures the next suggestion lookup returns fresh data with updated counts.

4.5 Frontend: Debouncing

Choice: 300ms debounce on the search input before firing API requests.

Without debouncing, typing "iphone" fires 6 requests: `i`, `ip`, `iph`, `ipho`, `iphon`, `iphone`. With 300ms debounce, it fires only 1 request — the final prefix after the user pauses typing.

Trade-off: Higher debounce delay = fewer requests but slower perceived responsiveness. 300ms is the industry standard — fast enough to feel instant, slow enough to eliminate intermediate keystrokes.

---

5. Performance Report

Measured via `GET /metrics` after simulating realistic traffic: 114 suggestion lookups across 19 unique prefixes and 50 search submissions across 5 unique queries.

5.1 Suggestion API Latency

Percentile	Latency
p50	1.10ms
p95	113.61ms
p99	116.20ms

Analysis:

- **p50 at 1.10ms** — the median request is served from Redis cache in about 1ms, well under any perceptible delay
- **p95/p99 at ~113ms** — these are cold-cache misses that hit PostgreSQL. The first request for each unique prefix causes a cache miss, queries the database, and populates the cache. All subsequent requests for the same prefix return in ~1ms
- In steady-state (warm cache with repeated prefixes), p95 drops to under 2ms

5.2 Cache Hit Rate

Metric	Value
Hits	95
Misses	19
Hit Rate	**83.33%**

Analysis:

- 19 misses correspond exactly to the 19 unique prefixes queried — each prefix causes exactly one miss on first lookup
- All 95 subsequent lookups for previously-seen prefixes were served from Redis cache
- In a production scenario with users repeatedly typing similar prefixes, the hit rate would approach 95-99%
- Cache entries expire after 60s (TTL), so a prefix not queried for 60s will cause one new miss on the next lookup

5.3 Batch Write Reduction

Metric	Value
Total Submissions	50
Flush Count	1
DB Writes	5
Writes Saved	45
Write Reduction	**90.00%**

Analysis:

- 50 search submissions across 5 unique queries were aggregated into a single flush containing 5 DB writes
- This represents a **90% reduction** in database write operations
- Example breakdown: 15 searches for "google chrome" became a single `UPDATE count = count + 15` — instead of 15 separate writes
- The batch writer used a single multi-row `INSERT ... ON CONFLICT DO UPDATE` statement, meaning only one database round-trip for all 5 writes

5.4 Overall System Latency

Percentile	Latency
p50	1.10ms
p95	112.37ms
p99	116.20ms

This includes all endpoint types (suggest, search, trending, cache debug, metrics). The POST /search endpoint responds in <1ms since it only buffers to memory without waiting for a DB write.

5.5 Summary

Metric	Value	Significance
Median suggestion latency	1.10ms	Sub-2ms response for cached lookups
Cache hit rate	83.33%	One miss per unique prefix, then all hits
Write reduction	90.00%	10x fewer DB writes through batching
Dataset size	1.24M unique queries	Real-world AOL search data
Cache nodes	3 (450 virtual nodes)	Even distribution via consistent hashing

The system achieves its design goals: fast prefix lookups via caching, reduced DB write pressure via batching, and relevant trending results via recency-aware ranking.